

Vishal Pandey | Applied ML Research

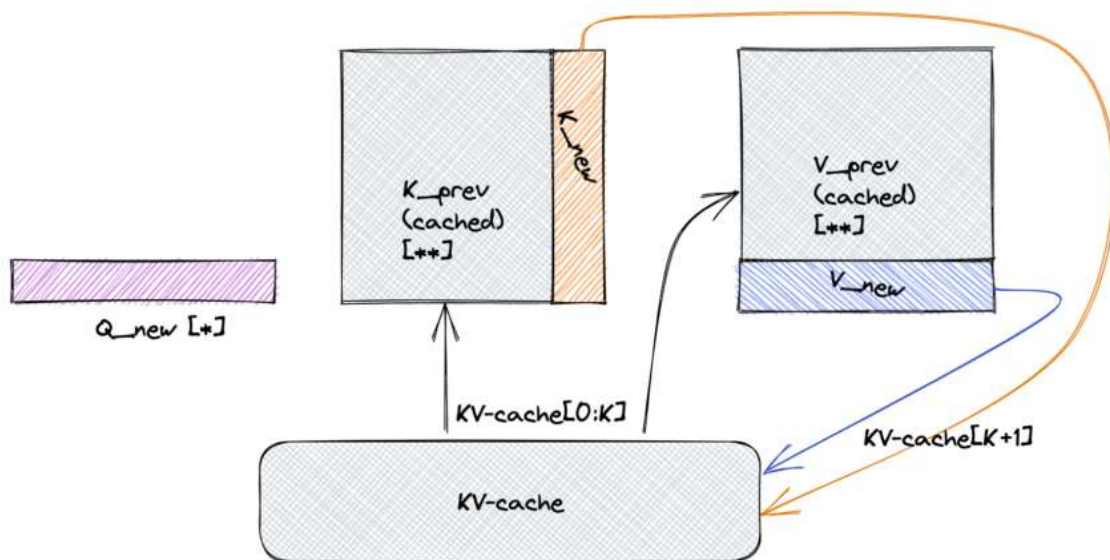
[Home](#) [About Me](#) [Blog](#)

KV Caching

18 Aug, 2025

A small note on the KV Caching. In short, it is the most powerful trick that makes LLM faster during inference.

Let's deep dive into this...



Notes:

- * When processing token[K], we only need the K'th row of Q
- ** When processing token[K], we require the full K & V tensors, but we can mostly reuse the cached values (This enables skipping the computation of K & V)

1. Where the problem comes from

At inference (say, autoregressive generation), you generate tokens one at a time:

Step 1: Input prompt → predict next token.

Step 2: Append predicted token → feed full sequence again → predict next.

Step 3: Repeat...

Naïvely, at each step you'd recompute attention across the entire prefix plus all past tokens. That's a computational cost of $O(T^2)$ as the sequence grows, way too

slow.

2. The Trick: Cache K, V Once

Attention formula:

$$\text{Attn}(Q_t, K_{1:t}, V_{1:t}) = \text{softmax}\left(\frac{Q_t K_{1:t}^\top}{\sqrt{d_k}}\right) V_{1:t}$$

Notice:

Keys $K_{1:t}$ and values $V_{1:t}$ come from all previous tokens (including the prefix).

At time $t + 1$, you don't need to recompute $K_{1:t}, V_{1:t}$. You already had them at step t .

You only need to compute K_{t+1}, V_{t+1} for the new token, then append them.

So each step just does:

- $K_{1:t+1} = [K_{1:t}; K_{t+1}]$
- $V_{1:t+1} = [V_{1:t}; V_{t+1}]$

Instead of reprocessing the whole history.

3. What's Actually Cached?

For every layer ℓ :

Store the matrices K^ℓ and V^ℓ for all tokens generated so far.

In practice, they're stored in shape:

$-(\text{batch_size}, \text{num_heads}, \text{seq_len}, d_k)$

- **batch_size**: number of sequences processed in parallel
- **num_heads**: number of attention heads
- **seq_len**: number of tokens seen so far
- d_k : dimensionality of each key vector per head

During generation:

- Forward pass for the new token computes only its Q, K, V .
 - Append new K, V to cache, use cached K, V for attention.
-

4. Why This Helps

- **Without caching:** recompute K, V for all t tokens at each step $\rightarrow O(T^2)$.
- **With caching:** compute K, V only once per token, then reuse $\rightarrow O(T)$.

This is why large LMs (like GPT) can generate long sequences in real time.

5. Prefix-Tuning Twist

In **Prefix-Tuning**, the prefix KV pairs (P_K, P_V) are **constant across steps** (they don't change with new tokens).

You can **pre-compute** them once per task and store them in the cache at the start.

At inference, they're **concatenated** with the cached KV from real tokens:

- $K_*^\ell = [P_K^\ell; K_{1:t}^\ell]$
- $V_*^\ell = [P_V^\ell; V_{1:t}^\ell]$

That means every query token can **always attend to both** prefix memory and past tokens without recomputing either.

So, **KV caching + prefix tuning = super efficient adaptation:**

- Prefix KV computed **once**
 - Token KV computed **incrementally**
-

6. KV Caching in a Nutshell

That's **KV caching** in detail:

- It stores **past keys and values per layer**
- Reuses them across decoding steps
- Makes **autoregressive inference** cost $O(T)$ instead of $O(T^2)$
- And in **Prefix-Tuning**, it makes prefixes basically "free" after the first compute

